

# Analog Matrix Inversion Circuit Design for Solving Tridiagonal Linear Systems: A Compact and Decoupled Approach

Sichun Du<sup>✉</sup>, Zhengmiao Wei<sup>✉</sup>, Pingdan Xiao<sup>✉</sup>, *Graduate Student Member, IEEE*, Yu Dong<sup>✉</sup>,  
Shiping Wen<sup>✉</sup>, *Senior Member, IEEE*, and Qinghui Hong<sup>✉</sup>

**Abstract**—Recently, analog matrix inversion circuits (INV) have demonstrated significant advantages in solving matrix equations. However, solving large-scale sparse tridiagonal linear systems (TLS) using full-scale INV introduces considerable zero regions redundancy and interconnection factors. To address this issue, this article proposes a dedicated INV for solving TLS. The TLS is reformulated into a tightly coupled matrix equation system (CMES), effectively eliminating the zero regions within the array of INV. Very large-scale TLS (VLTLS) are processed by the proposed decoupled INV equipped with a spike algorithm engine. Compared with the general INV, the proposed circuit offers enhanced advantages in compactness and scalability. PSpice-based simulation experiments demonstrate that the proposed circuit achieves a solving accuracy of over 99%. Comprehensive nonidealities are also evaluated, with a curve-fitting accuracy of 98.41%. Utilizing commercial operational amplifier (OA) models, the proposed circuit achieves microsecond-level computation speed, offering a two-order-of-magnitude acceleration compared to conventional software-based approaches. Furthermore, the solution of a 1-D diffusion equation with 1024 internal nodes verifies the effectiveness of the proposed circuit in solving large-scale TLS.

**Index Terms**—Analog computing, circuit design, memristor, scalability, sparse matrix inversion.

## I. INTRODUCTION

**T**RIDIAGONAL linear systems (TLS), as a special type of linear system, serve as a unified framework for various problems in scientific computing, such as the finite-difference method for diffusion equations [1], boundary value problems of ordinary differential equations [2], and the computation of

cubic spline functions [3]. Such techniques find applications in engineering problems involving fluid dynamics, thermodynamics, power system analysis, and motion planning [4], [5], [6], [7], [8], [9]. Efficiently solving TLS is, therefore, critical to addressing these problems [10].

Executing traditional algorithms (e.g., direct methods [11] or iterative methods [12]) on general-purpose processors is a mature and stable technique for solving matrix equations. Nevertheless, the dramatic increase in complexity caused by high-dimensional matrix equations remains a major bottleneck for these traditional methods. Recent studies show that analog matrix computing (AMC) effectively reduces the burden on digital computers in processing matrix data [13]. The contributions of AMC to solving matrix equations can be summarized as analog iteration [14], [15] and analog matrix inversion [16], [17], [18], [19], [20]. Analog iteration maps iterative algorithms (e.g., the Jacobi method [21]) onto AMC, where AMC accelerates the matrix–vector multiplication (MVM) involved in the iteration. In contrast, an analog matrix inversion circuit (INV) typically does not involve iterative computation but solves matrix equations in one step. By leveraging the massive spatial parallelism of the cross-point arrays and the concurrent feedback mechanism in the circuit, the time complexity of solving an  $n$ -dimensional matrix equation was reduced from  $O(n^3)$  to  $O(1)$  [22].

In principle, existing INV techniques are potential candidates for solving TLS. However, the coefficient matrix of TLS has a typical banded structure, with most of its elements being zeros [23]. In cross-point arrays, representing a single zero value requires one [24] or even two [25] devices. This means that mapping the coefficient matrix of TLS onto a full-sized cross-point array results in a large amount of wasted space [26]. This phenomenon becomes more pronounced as the matrix dimension increases. In addition, some potential nonideal factors may increase (e.g., interconnection resistance [27] and device variation [28]). A comprehensive consideration of both large-scale and sparse characteristics in TLS is still lacking.

The scalability of AMC is a nonnegligible problem. Based on the Schur complement algorithm [29], Pan et al. [19] proposed a scalable AMC (BlockAMC). This scheme partitions the original large matrix across different memory arrays, where INV and MVM operations are executed to progressively

Received 9 June 2025; revised 13 August 2025; accepted 15 September 2025. Date of publication 9 October 2025; date of current version 27 November 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62371186, Grant 62234008, and Grant 62471177; in part by the Huxiang Young Talents Program under Grant 2023RC3103; in part by the Key Research and Development Program of Hunan Province under Grant 2024JK2009; and in part by the Hunan Postgraduate Scientific Research Innovation Project under Grant CX20240421. (Corresponding author: Qinghui Hong.)

Sichun Du, Zhengmiao Wei, Pingdan Xiao, Yu Dong, and Qinghui Hong are with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 418002, China (e-mail: jt\_dsc@hnu.edu.cn; weizhengmiao@hnu.edu.cn; xiaopd@hnu.edu.cn; yudongltg@hnu.edu.cn; hongqinghui@hnu.edu.cn).

Shiping Wen is with the Australian AI Institute, Faculty of Engineering and Information Technology, University of Technology Sydney, Sydney, NSW 2007, Australia (e-mail: shiping.wen@uts.edu.au).

Digital Object Identifier 10.1109/TVLSI.2025.3612422

1063-8210 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

obtain partial solutions and reconstruct the original solution. This innovative method makes a key contribution toward resolving the infamous scalability issue of INV. Unfortunately, since the Schur complement algorithm does not take into account the sparsity of TLS during matrix domain decomposition, a large number of zero elements are preserved. This makes the deployment of TLS in BlockAMC inefficient and resource-consuming. Moreover, TLS typically aim to decompose into submatrices as small as possible. Achieving this in BlockAMC requires recursive partitioning and multilevel interconnections. This, in turn, reduces the parallel computing advantage of AMC. On the other hand, analog iteration [14], [15] maps a large sparse matrix into multiple MVM units, with peripheral adder circuits used to compute the global solution. This method is simpler, but it is only applicable to MVM and not suitable for INV. The search for more efficient algorithms to bridge the compatibility gap between TLS and AMC warrants renewed attention.

Motivated by the above discussion, this article proposes a dedicated INV tailored for TLS exhibiting large-scale sparsity. The specific work of this article begins with a basic block circuit for TLS deployment. The TLS is formulated as a tightly coupled matrix equation system (CMES) with only the nonzero regions retained. It is further demonstrated that removing the zero regions does not directly affect circuit stability. Subsequently, the basic block circuit is used to construct a decoupled circuit for solving very large-scale TLS (VLTLS). The VLTLS can be partitioned and processed on demand within limited hardware space. Finally, key functional tests are conducted on the proposed circuit, covering feasibility analysis and operational performance under nonideal conditions. Its effectiveness is further validated through the application to a 1-D diffusion equation. The main contributions are summarized as follows.

- 1) A high-speed TLS solving method is proposed, achieving solution times at the microsecond level.
- 2) A zero regions elimination mapping scheme within the INV is introduced, providing a more compact circuit structure that effectively addresses the inherent sparsity of TLS.
- 3) A circuit design methodology is developed to support flexible decoupling and parallel scalability for large-scale TLS problems, with a comprehensive evaluation of the impact of nonideal factors on solution accuracy.

The rest of this article is organized as follows. Section II introduces the design logic and operating principles of the proposed circuit. Section III provides simulation results and analysis of the proposed circuit. Section IV discusses the practicality of the proposed circuit using the 1-D diffusion equation as an example. Finally, this article is concluded in Section V.

**Notation:** All numbers in this article are real.  $A_{n \times m}$ ,  $a_{n \times 1}$ , and  $a_{1 \times n}$  denote an  $n \times m$  matrix, an  $n$ -dimensional column vector, and an  $n$ -dimensional row vector, respectively. For a matrix (vector)  $A$  ( $b$ ),  $A_{ij}$  ( $b_i$ ) is its  $i$ th row and  $j$ th column block matrix, and  $A_{xy}$  ( $b_{x,y}$ ) are its  $x$ th row and  $y$ th column elements. Given  $X = [x_1 \ x_2 \ \dots \ x_n]$ ,  $\text{vec}(X) = [x_1^T \ x_2^T \ \dots \ x_n^T]^T$ . Any matrix (vector)  $C$  ( $d$ )

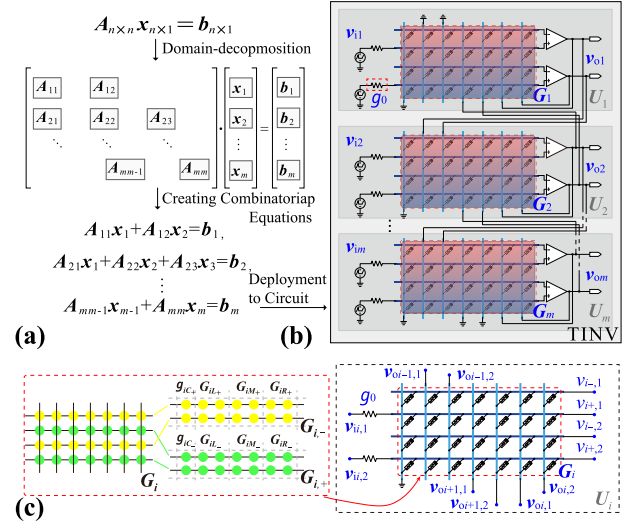


Fig. 1. Basic block-solution circuit design for TLS. (a) Transformation of the TLS into a CMES. (b) Circuit implementation of the CMES. (c) Encoding of the cross-point resistive memristor arrays and the key voltage nodes.

can be expressed as the difference of two nonnegative ones:  $C = C_+ - C_-$  ( $d = d_+ - d_-$ ). For integers  $m \leq l$ , the index range  $j[m, l]$  represents the set  $j = (m, m+1, \dots, l)$ . Elements beyond the defined range are set to zero. For example,  $A_{ii-1} = 0$  when  $i = 1$ .

## II. CIRCUIT DESIGN FOR SOLVING TLS

### A. Basic Block-Solution Circuit Design for TLS

Consider an arbitrary linear system with appropriately sized (order of coefficient matrix  $A$ )

$$A_{n \times n} x_{n \times 1} = b_{n \times 1} \quad (1)$$

where  $A$  is a tridiagonal matrix of the following form [29]:

$$A = \begin{bmatrix} a_{11} & a_{12} & & & \\ a_{21} & a_{22} & a_{23} & & \\ \ddots & \ddots & \ddots & \ddots & \\ & & a_{n-1n-2} & a_{n-1n-1} & a_{n-1n} \\ & & & a_{nn-1} & a_{nn} \end{bmatrix}. \quad (2)$$

This section proposes a region partitioning and equation restructuring approach to reduce hardware overhead, as shown in Fig. 1(a). Domain decomposition for system (1) is performed as follows.  $x_j$  and  $b_j$  are each of order  $2 \times 1$ , and  $A_{ij}$  is  $2 \times 2$ . Specifically, the coefficient matrix  $A$  retains only the submatrices that contain nonzero elements. For a nonsymmetric structure, some zero elements are included for coding convenience to maintain structural symmetry. As a result, a CMES containing  $m$  matrix equations is obtained.

In Fig. 1(b), we present a tridiagonal matrix INV (TINV) based on a conductance compensation matrix INV [20]. TINV is divided into  $m$  subregions. Fig. 1(c) shows the encoding of the cross-point resistive memristor array and the key voltage nodes in subregion  $U_i$ ,  $i[1, m]$ . The cross-point resistive memristor array is represented by the matrix  $G_i$ . Rows in  $G_i$  connected to the inverting input of the

According to Kirchhoff's current law and Ohm's law. Voltage at the inverting input

$$\begin{aligned}
&= \left[ \Sigma_{j=1}^2 \left( \mathbf{G}_{iL+,kj} \mathbf{y}_{oi-1,j} + \mathbf{G}_{iM+,kj} \mathbf{y}_{oi+1,j} + \mathbf{G}_{iR+,kj} \mathbf{y}_{oi,j} \right) \right] \\
&\quad / \left[ \Sigma_{j=1}^2 \left( \mathbf{G}_{iL+,kj} + \mathbf{G}_{iM+,kj} + \mathbf{G}_{iR+,kj} \right) + \mathbf{g}_{iC+,k} \right] \quad (3)
\end{aligned}$$

and the voltage at the noninverting input

$$v_{i+,k} = \left[ \sum_{j=1}^2 \left( \mathbf{G}_{iL-,kj} \mathbf{v}_{oi-1,j} + \mathbf{G}_{iM-,kj} \mathbf{v}_{oi+1,j} + \mathbf{G}_{iR-,kj} \mathbf{v}_{oi,j} \right) + g_0 \mathbf{v}_{ii,k} \right] / \left[ \sum_{j=1}^2 \left( \mathbf{G}_{iL-,kj} + \mathbf{G}_{iM-,kj} + \mathbf{G}_{iR-,kj} \right) + g_0 + \mathbf{g}_{iC-,k} \right]. \quad (4)$$

Adjust the value of the compensating conductance to make

$$\mathbf{g}_{iC-,k} - \mathbf{g}_{iC+,k} = -\mathbf{g}_0 + \sum_{j=1}^2 (\mathbf{G}_{iL,kj} + \mathbf{G}_{iM,kj} + \mathbf{G}_{iR,kj}). \quad (5)$$

At this point, from  $v_{ik+} - v_{ik-} = 0$  gives

$$\sum_{j=1}^2 (\mathbf{G}_{iL,kj} \mathbf{v}_{oi-1,j} + \mathbf{G}_{iM,kj} \mathbf{v}_{oi+1,j} + \mathbf{G}_{iR,kj} \mathbf{v}_{oi,j}) = g_0 \mathbf{v}_{ii,k}. \quad (6)$$

Therefore, for all  $k$ -guided rows, the relationship can be expressed using the following matrix equation:

$$\mathbf{G}_{iL}\mathbf{v}_{oi-1} + \mathbf{G}_{iM}\mathbf{v}_{oi+1} + \mathbf{G}_{iR}\mathbf{v}_{oi} = g_0\mathbf{v}_{ii}. \quad (7)$$

Furthermore, considering all  $U_i$  in TINV, they form a coupled system, namely

$$\begin{cases} \mathbf{G}_{1R}\mathbf{v}_{o1} + \mathbf{G}_{1M}\mathbf{v}_{o2} = g_0\mathbf{v}_{i1} \\ \mathbf{G}_{2L}\mathbf{v}_{o1} + \mathbf{G}_{2R}\mathbf{v}_{o2} + \mathbf{G}_{2M}\mathbf{v}_{o3} = g_0\mathbf{v}_{i2} \\ \vdots \\ \mathbf{G}_{mL}\mathbf{v}_{om-1} + \mathbf{G}_{mR}\mathbf{v}_{om} = g_0\mathbf{v}_{im}. \end{cases} \quad (8)$$

Here, the mathematical formulation of the CMES has been fully reconstructed in TINV. The  $i$ th subequation in the CMES is mapped to  $U_i$  in TINV. Using the voltage factor  $v_0$  and the conductance factor  $g_0$ , the data in the CMES is deployed as physical quantities with dimensions in the circuit. Specifically, the mapping method of the CMES created in Fig. 1(a) onto the TINV [Fig. 1(b)] is given by  $\mathbf{v}_{ii} = v_0 \mathbf{b}_i$ ,  $\mathbf{v}_{oi} = v_0 \mathbf{x}_i$ ,  $\mathbf{G}_{iL} = g_0 \mathbf{A}_{ii-1}$ ,  $\mathbf{G}_{iM} = g_0 \mathbf{A}_{ii+1}$ , and  $\mathbf{G}_{iR} = g_0 \mathbf{A}_{ii}$ . Clearly, at this point, (8) can provide the solution to system (1) as  $\mathbf{x} = \text{vec}([\mathbf{v}_{o1} \ \mathbf{v}_{o2} \ \cdots \ \mathbf{v}_{om}]) / v_0$ . During the operation of the circuit, the conductance matrix is preloaded, after which the input voltage is applied to produce the computation results.

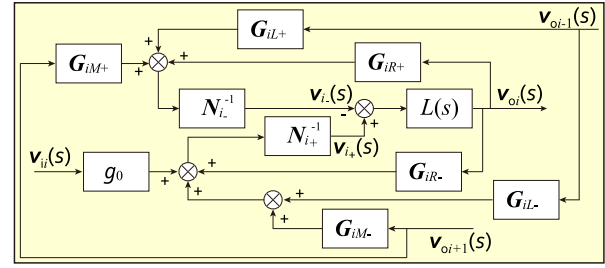


Fig. 2. Simplified system block diagram corresponding to  $U_i$ .

### B. Stability Analysis of the TINV Circuit

Based on (3) and (4), a simplified system block diagram of  $U_i$  is given as shown in Fig. 2 when the TINV circuit solves system (1).  $L(s)$  is the open-loop gain of the OAs,  $s$  is the complex variable in the Laplace transform.  $N_{i-} = \text{diag}(\mathbf{G}_{iC+,1} + \sum_{j=1}^2 (\mathbf{G}_{iL+,1j} + \mathbf{G}_{iM+,1j} + \mathbf{G}_{iR+,1j}), \mathbf{G}_{iC+,2} + \sum_{j=1}^2 (\mathbf{G}_{iL+,2j} + \mathbf{G}_{iM+,2j} + \mathbf{G}_{iR+,2j}))$ .  $N_{i+} = \text{diag}(g_0 + \mathbf{G}_{iC-,1} + \sum_{j=1}^2 (\mathbf{G}_{iL-,1j} + \mathbf{G}_{iM-,1j} + \mathbf{G}_{iR-,1j}), g_0 + \mathbf{G}_{iC-,2} + \sum_{j=1}^2 (\mathbf{G}_{iL-,2j} + \mathbf{G}_{iM-,2j} + \mathbf{G}_{iR-,2j}))$ . The signal transmission relationships analyzed from Fig. 2 are as follows:

$$\begin{aligned} \mathbf{v}_{i-}(s) &= \mathbf{N}_{i-}^{-1} \left( \mathbf{G}_{iL+} \mathbf{v}_{oi-1}(s) + \mathbf{G}_{iR+} \mathbf{v}_{oi}(s) \right. \\ &\quad \left. + \mathbf{G}_{iM+} \mathbf{v}_{oi+1}(s) \right) \\ \mathbf{v}_{i+}(s) &= \mathbf{N}_{i+}^{-1} \left( \mathbf{G}_{iL-} \mathbf{v}_{oi-1}(s) + \mathbf{G}_{iR-} \mathbf{v}_{oi}(s) \right. \\ &\quad \left. + \mathbf{G}_{iM-} \mathbf{v}_{oi+1}(s) + g_0 \mathbf{v}_{ii}(s) \right) \end{aligned} \quad (9)$$

and

$$L(s) \left( \mathbf{v}_{i+}(s) - \mathbf{v}_{i-}(s) \right) = \mathbf{v}_{oi}(s). \quad (10)$$

So, the coupled system formed by all  $U_i$  has the following relations:

$$L(s) \left( N_+^{-1} (\mathbf{G}_N \mathbf{v}_o(s) + g_0 \mathbf{v}_i(s)) - N_-^{-1} \mathbf{G}_P \mathbf{v}_o(s) \right) = \mathbf{v}_o(s) \quad (11)$$

$$\begin{aligned} \text{where } \mathbf{v}_o(s) &= \text{vec}(\mathbf{v}_{o1}(s) \ \mathbf{v}_{o2}(s) \ \cdots \ \mathbf{v}_{om}(s)), \mathbf{v}_i(s) = \\ &\text{vec}(\mathbf{v}_{i1}(s) \ \mathbf{v}_{i2}(s) \ \cdots \ \mathbf{v}_{im}(s)), \quad \mathbf{N}_+^{-1} = \\ &\text{diag}(\mathbf{N}_{1+}^{-1}, \dots, \mathbf{N}_{m+}^{-1}), \mathbf{N}_-^{-1} = \text{diag}(\mathbf{N}_{1-}^{-1}, \dots, \mathbf{N}_{m-}^{-1}) \end{aligned}$$

$$\mathbf{G}_N = \begin{bmatrix} \mathbf{G}_{1R_-} & \mathbf{G}_{1M_-} & & \\ & \ddots & & \\ & & \mathbf{G}_{mL_-} & \mathbf{G}_{mR_-} \end{bmatrix} \quad (12)$$

and

$$\mathbf{G}_P = \begin{bmatrix} \mathbf{G}_{1R+} & \mathbf{G}_{1M+} & & \\ & \ddots & & \\ & & \mathbf{G}_{mL+} & \mathbf{G}_{mR+} \end{bmatrix}. \quad (13)$$

Further, considering  $\mathbf{G}_N = g_0 \mathbf{A}_-$ ,  $\mathbf{G}_P = g_0 \mathbf{A}_+$ ,  $\mathbf{v}_o(s) = v_0 \mathbf{x}(s)$ ,  $\mathbf{v}_i(s) = v_0 \mathbf{b}(s)$ ,  $\mathbf{N}'_+^{-1} = \mathbf{N}_+^{-1}/g_0$  and  $\mathbf{N}'_-^{-1} = \mathbf{N}_-^{-1}/g_0$ . Equation (11) becomes

$$L(s) \left( N_+^{\prime -1} (A_- \mathbf{x}(s) + \mathbf{b}(s)) - N_-^{\prime -1} A_+ \mathbf{x}(s) \right) = \mathbf{x}(s) \quad (14)$$

that is

$$\begin{aligned} \mathbf{x}(s) &= \mathbf{H}(s) \mathbf{b}(s) \\ \mathbf{H}(s) &= \left( \frac{1}{L(s)} \mathbf{I} + \mathbf{N}'^{-1}_- \mathbf{A}_+ - \mathbf{N}'^{-1}_+ \mathbf{A}_- \right)^{-1} \mathbf{N}'^{-1}_+. \end{aligned} \quad (15)$$

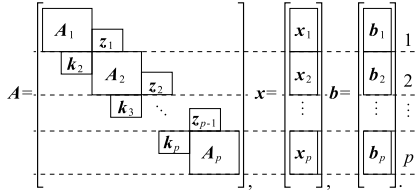
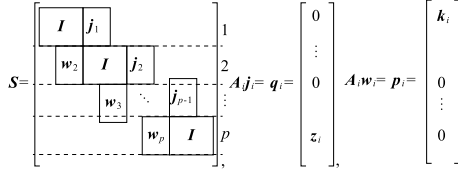

 Fig. 3. Partitioning of the systems (1) with  $p$  partitions.


Fig. 4. Spike matrix.

For a single-pole OA model, transfer function  $L(s) = (L_0/(1 + (s/\omega_0)))$ , where  $L_0$  is the dc open-loop gain and  $\omega_0$  is the 3-dB bandwidth. Bringing in  $L(s)$  yields  $\mathbf{H}(s)$

$$= \left( \frac{1}{L_0} \mathbf{I} + \frac{s}{L_0 \omega_0} \mathbf{I} + \mathbf{N}'^{-1} \mathbf{A}_+ - \mathbf{N}'^{-1} \mathbf{A}_- \right)^{-1} \mathbf{N}'^{-1}. \quad (16)$$

Since  $L_0$  is much larger than 1, the  $(1/L_0)\mathbf{I}$  term in  $\mathbf{H}(s)$  is omitted. Defining  $\mathbf{M} = \mathbf{N}'^{-1} \mathbf{A}_+ - \mathbf{N}'^{-1} \mathbf{A}_-$ , calculating  $\det((s/L_0 \omega_0)\mathbf{I} + \mathbf{M}) = 0$  gives the characteristic roots of the system as  $s_i = -L_0 \omega_0 \lambda_{Mi}$ . When all eigenvalues  $\lambda_{Mi}$  of  $\mathbf{M}$  are positive, each  $s_i$  has a negative real part, making the system stable. Indeed, the stability result of TINV is similar to that of the conductance compensation matrix INV [20]. This indicates that the stability of TINV is governed by the original matrix  $\mathbf{A}$  in system (1) and is not affected by domain decomposition.

### C. Solving VLTLs

The physical constraints of the circuit architecture limit the mapping of VLTLs [30]. This section employs the SPIKE algorithm [31] to decouple VLTLs, breaking down large-scale computations into appropriately sized computational domains. The procedure involves the following steps.

- Step 1: Divide system (1) according to the rules in Fig. 3.  $\mathbf{A}$  is decomposed into invertible block diagonal subtridiagonal matrices  $\mathbf{A}_{j(n/p) \times (n/p)}$ ,  $j[1, p]$ , superdiagonal matrix blocks  $\mathbf{z}_{j1 \times 1}$ ,  $j[1, p-1]$ , and subdiagonal matrix blocks  $\mathbf{k}_{j1 \times 1}$ ,  $j[2, p]$ . Meanwhile,  $\mathbf{x}$  and  $\mathbf{b}$  are also divided into  $p$  groups, consisting of  $\mathbf{x}_{j1 \times p}$ ,  $j[1, p]$ , and  $\mathbf{b}_{j1 \times p}$ ,  $j[1, p]$ , respectively.
- Step 2: Let  $\mathbf{A}$  be factorized as  $\mathbf{A} = \mathbf{D}\mathbf{S}$ , where  $\mathbf{D} = \text{diag}(\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_p)$  and  $\mathbf{S}$  is the spike matrix illustrated in Fig. 4.
- Step 3: The postprocessing stage transforms the original problem (1), as outlined in Step 2, into two subproblems

$$\mathbf{D}\mathbf{y} = \mathbf{b} \quad (17)$$

and

$$\mathbf{S}\mathbf{x} = \mathbf{y}. \quad (18)$$

$$\hat{\mathbf{S}} = \begin{bmatrix} 1 & 0 & \mathbf{j}_1^{(t)} & & \\ 0 & 1 & \mathbf{j}_1^{(b)} & & \\ & & \ddots & & \\ \mathbf{w}_2^{(t)} & & 1 & 0 & \mathbf{j}_{p-1}^{(t)} \\ \mathbf{w}_2^{(b)} & & 0 & 1 & \mathbf{j}_{p-1}^{(b)} \\ & & & & \ddots \\ & & \mathbf{w}_p^{(t)} & & 1 & 0 \\ & & \mathbf{w}_p^{(b)} & & 0 & 1 \end{bmatrix}, \quad \hat{\mathbf{x}} = \begin{bmatrix} \mathbf{x}_1^{(t)} \\ \mathbf{x}_1^{(b)} \\ \vdots \\ \mathbf{x}_p^{(t)} \\ \mathbf{x}_p^{(b)} \end{bmatrix}, \quad \hat{\mathbf{y}} = \begin{bmatrix} \mathbf{y}_1^{(t)} \\ \mathbf{y}_1^{(b)} \\ \vdots \\ \mathbf{y}_p^{(t)} \\ \mathbf{y}_p^{(b)} \end{bmatrix}.$$

Fig. 5. Simplified (18).

In each partitioned region, the top and bottom rows are denoted as  $(t)$  and  $(b)$ , respectively, and the middle portion as  $(r)$ . Further processing of (18) yields

$$\hat{\mathbf{S}}\hat{\mathbf{x}} = \hat{\mathbf{y}}. \quad (19)$$

It consists of the top and bottom rows of each partition in (18), shown in Fig. 5. After extracting the boundary rows from (18), the resulting simplified (19) has a size of only twice the number of groups  $p$ .

Finally, the global solution  $\mathbf{x}$  is reconstructed with perfect parallelism from  $\mathbf{x}_k^{(b)}$ ,  $k[1, p-1]$ , and  $\mathbf{x}_k^{(t)}$ ,  $k[2, p]$  as follows:

$$\begin{cases} \mathbf{x}_1^{(r)} = \mathbf{y}_1^{(r)} - \mathbf{j}_1^{(r)} \mathbf{x}_2^{(t)} \\ \mathbf{x}_i^{(r)} = \mathbf{y}_i^{(r)} - \mathbf{j}_i^{(r)} \mathbf{x}_{i+1}^{(t)} - \mathbf{w}_i^{(r)} \mathbf{x}_{i-1}^{(b)} \\ \mathbf{x}_p^{(r)} = \mathbf{y}_p^{(r)} - \mathbf{w}_p^{(r)} \mathbf{x}_{p-1}^{(b)}. \end{cases} \quad (20)$$

In summary, the SPIKE algorithm decomposes the large-scale tridiagonal matrix inversion problem  $\mathbf{A}\mathbf{x} = \mathbf{b}$  into multiple matrix inversion subproblems: 1)  $\mathbf{A}_i \mathbf{j}_i = \mathbf{q}_i$ ; 2)  $\mathbf{A}_i \mathbf{w}_i = \mathbf{p}_i$ ; 3)  $\mathbf{D}\mathbf{y} = \mathbf{b}$ ; 4)  $\hat{\mathbf{S}}\hat{\mathbf{x}} = \hat{\mathbf{y}}$ , along with multiple matrix multiplication and matrix addition operations as given in (20). The subproblem  $\mathbf{D}\mathbf{y} = \mathbf{b}$  consists of multiple smaller matrix inversion problems; and 5)  $\mathbf{A}_i \mathbf{y}_i = \mathbf{b}_i$ .

The VLTLs solving circuit (VLTINV), designed based on the SPIKE algorithm, is shown in Fig. 6. It consists of a front-end inversion (FINV) module, a postprocessing inversion (PINV) module, a solution reconstruction (SRE) module, a multiplexer, a sample and hold, a programming module, a controller, a global input, a local output, and a global output.

The FINV includes  $f$  TINVs and receives global input signals, handling subproblems 1), 2), and 5). Since 1), 2), and 5) share the same coefficient matrix  $\mathbf{A}_i$ . Once FINV is loaded with  $\mathbf{A}_i$ , the corresponding solution vectors  $\mathbf{j}_i$ ,  $\mathbf{w}_i$ , and  $\mathbf{y}_i$  can be obtained by sequentially inputting the right-hand side vectors  $\mathbf{q}_i$ ,  $\mathbf{p}_i$ , and  $\mathbf{b}_i$  for 1), 2), and 5), respectively. The output of FINV ultimately flows to the local output and the multiplexer.

The multiplexer consists of two rows of switching gates, directing the flow of FINV output based on time segments. Specifically, the transmission gates  $N$  are on, and the transmission gates  $P$  are off when the global input is in phases I to IV. This process solves the submatrices  $\mathbf{j}_i$  and  $\mathbf{w}_i$  of the matrix  $\mathbf{S}$  and loads them into the cross-point resistive memristor array in the PINV and SRE. We use the programming module proposed in previous work to map voltage values to conductance values [32]. After that, the transmission gates  $N$  are turned off, and the transmission gates  $P$  are turned on. The output of global

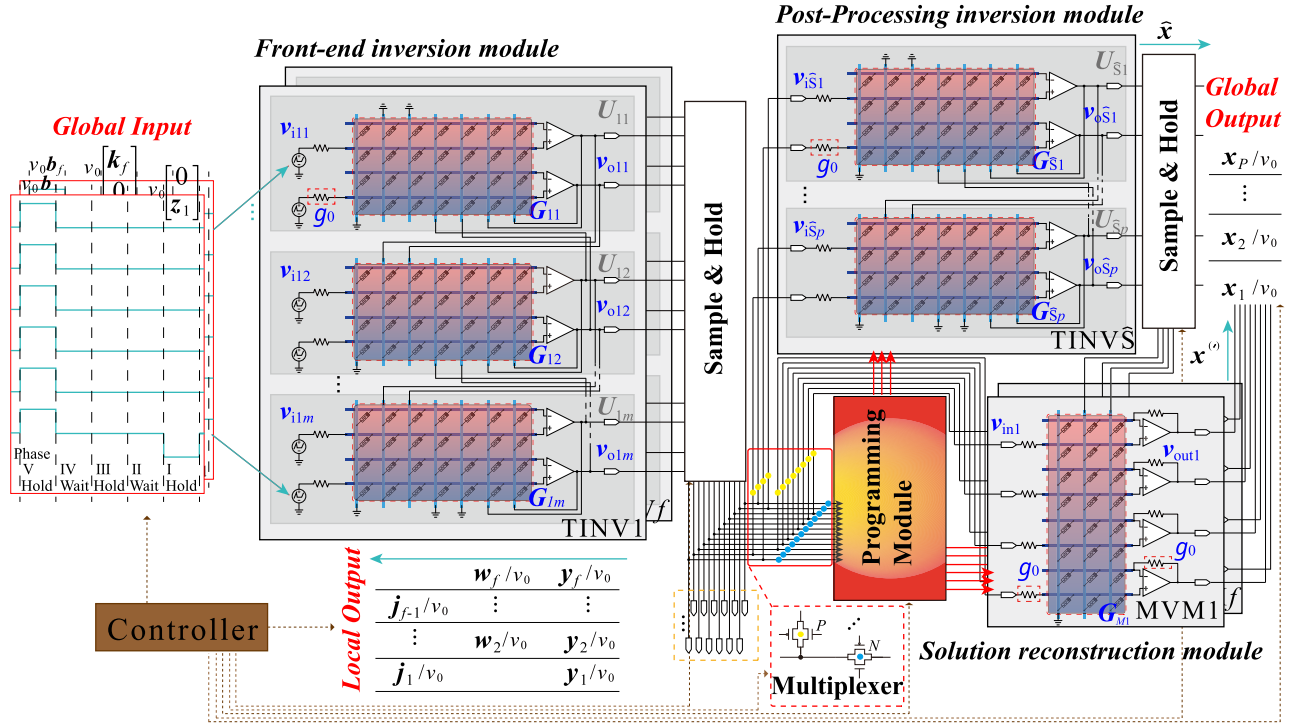


Fig. 6. VLTLS solution circuit (VLTINV).

input that is in phases V, that is, the solution  $y_i$  of 5), is fed into PINV and SRE. The PINV takes a portion  $v_0 \hat{y}$  of  $v_0 y$ , the output of FINV, as input.  $\hat{S}$  is a five-diagonal matrix. Its implementation in TINVS is identical to that of a tridiagonal matrix, because the five diagonals of the original matrix  $A$  are fully preserved after domain decomposition in Fig. 1(a). Therefore, PINV can be used to solve the subproblem 4), with the output  $v_{o\hat{s}} = v_0 \hat{x}$ .  $v_{o\hat{s}}$  simultaneously flows to both the global output and the SRE.

The SRE is designed to consist of  $p$  parallel MVM modules. The  $i$ th MVM (MVM $i$ ) is responsible for computing  $x_i^{(r)}$  in subproblem (20), with  $v_{ini} = v_0 y_i^{(r)}$ ,  $G_{Mi,-} = g_0[c_{Mi,+}^{(r)} j_{i+}^{(r)} w_i^{(r)}]$ ,  $G_{Mi,+} = g_0[c_{Mi,-}^{(r)} j_i^{(r)} w_i^{(r)}]$ ,  $G_{Mi} = G_{Mi,-} - G_{Mi,+}$ , and  $v_{outi} = v_0 x_i^{(r)}$ . Referring to (3) and (4), when the compensated conductance vector satisfies

$$c_{Mi,-}^{(r)} - c_{Mi,+}^{(r)} = j_{i,kj}^{(r)} + w_{i,kj}^{(r)} \quad (21)$$

then

$$x_i^{(r)} = y_i^{(r)} - j_i^{(r)} x_{i+1}^{(r)} - w_i^{(r)} x_{i-1}^{(r)}. \quad (22)$$

Thus, all MVMs in the SRE provide the full computation results of subproblem (20). Finally,  $x_i^{(r)}$  and  $\hat{x}$  converge at the global output, forming the solution  $x$  of the original VLTLS problem  $Ax = b$ .

### III. SIMULATION AND PERFORMANCE EVALUATION OF THE PROPOSED CIRCUIT

This section discusses the feasibility of the proposed circuit. The impact of nonideal factors on the circuit is analyzed, including interconnection resistance, device variations, and noise. The area and power consumption of the proposed

circuit are assessed. The advantages of the proposed circuit in addressing large-scale sparse TLS are discussed. The experimental setup was as follows. The processor's specifications for running the application were as follows: the RAM size was 15.7 GB, the CPU model was 13th Gen Intel<sup>1</sup> Core<sup>2</sup> i5-13400 with 16 CPU cores, and the CPU speed was 2.50 GHz. The GPU used is an NVIDIA GeForce RTX 3060. The reference voltage  $v_0$  was set to 1 V, and the conversion conductance  $g_0$  was set to 100  $\mu$ S. The memristor adopted the drift speed adaptive memristor model [33]. A real OA model, AD823 from Analog Devices, with a GBWP of 16 MHz, was used.

#### A. Feasibility of the Proposed Circuit

This section first evaluates the solution accuracy of TINV under ideal conditions, that is, without considering nonideal factors, and then presents its circuit implementation based on physical hardware. A large number of systems (1) are generated using MATLAB's random functions and solved on the PSpice simulation platform using the proposed circuit. Coefficient matrices  $A$  of four different sizes are considered:  $16 \times 16$ ,  $32 \times 32$ ,  $64 \times 64$ , and  $128 \times 128$ . The calculation of the average solution accuracy is as follows:

$$\frac{1}{n} \times \sum_{j=1}^n \left( 1 - \frac{|x_{j,j} - x_{j,j}^*|}{|x_{j,j}|} \right) \times 100\% \quad (23)$$

where  $x_{n \times 1}$  is the analytical solution (Ideal value) provided by MATLAB, and  $x_{n \times 1}^*$  is the solution obtained using TINV (Test value).

<sup>1</sup>Registered trademark.

<sup>2</sup>Trademarked.

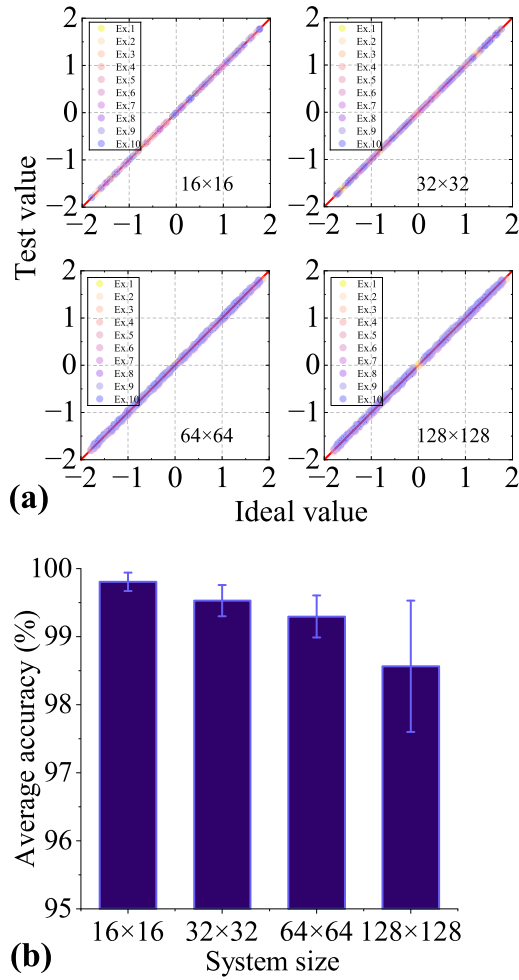


Fig. 7. Solution accuracy of TINV under ideal conditions. (a) Comparison between circuit output and ideal result. (b) Statistics of the average solution accuracy.

As shown in Fig. 7(a), ten experiments were conducted on system (1) for each of the four matrix sizes. The results indicate that the circuit output converges effectively to the analytical solution. However, the convergence performance shows a slight degradation as the size of the coefficient matrix increases. The bar chart in Fig. 7(b) provides a more detailed representation of the accuracy, including the mean and standard deviation of multiple experimental results. The mean of the average accuracy for the experiments across all four matrix sizes is above 98%. As the matrix size increases, nonideal factors in the circuit also increase with the expansion of the circuit [34], leading to a reduction in the output accuracy. Specifically, when the matrix order increases from 16 to 32, 64, and 128, the average accuracy decreases by 0.318%, 0.512%, and 1.245%, respectively. However, even when the matrix order reaches 128, the average solution accuracy of the circuit remains close to 98%, at  $98.56\% \pm 0.96\%$ .

Due to cost and experimental constraints, the memristor is replaced by a variable resistor in this experiment. Some functional tests require manual wiring connections. As shown in Fig. 8, the experimental setup includes variable resistor arrays (3296W-1-203LF,  $20\text{ k}\Omega \pm 10\%$ ), OAs (HT741ANZ),

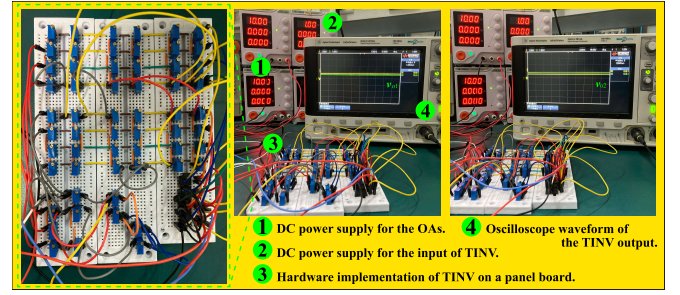


Fig. 8. Small-scale breadboard hardware experiment as a proof-of-concept demonstration of TINV.

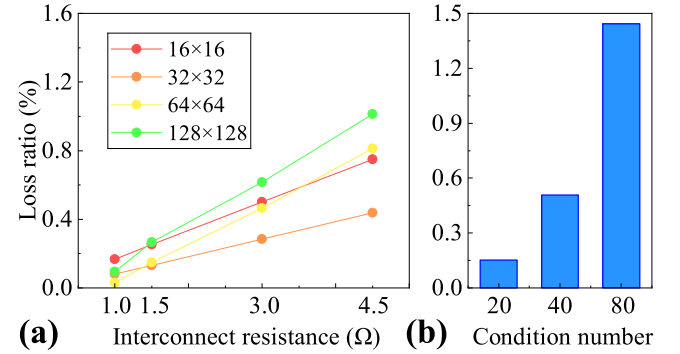


Fig. 9. (a) Average accuracy loss ratio of TINV solving system (1) with fixed size and condition number under different interconnect resistance values. (b) Average accuracy loss ratio of TINV under interconnect resistance ( $1.5\text{ }\Omega$ ) for the system (1) of the same size ( $32 \times 32$ ) with different condition numbers.

dc voltage sources, and an oscilloscope. It should be noted that using OAs with different parameters may lead to variations in circuit computation [22]. From a cost perspective, the HT741ANZ is selected for use in small-scale preliminary hardware validation experiments.

Specifically, a  $4 \times 4$  system (1) with the true solution  $\mathbf{x} = [0.6\ 0.8\ 0.8\ 0.6]^T$  is tested. According to the mapping rules in Section II-A, deploy this system on the breadboard. A stable output is observed on the oscilloscope from the breadboard. The results obtained by sampling within the steady-state interval are as  $\mathbf{v}_{o1} = [0.59\ 0.79]^T\text{V}$  and  $\mathbf{v}_{o2} = [0.79\ 0.60]^T\text{V}$ . That is, the circuit provides an output solution  $\mathbf{x}^* = [0.59\ 0.79\ 0.79\ 0.60]^T$ , which corresponds to an average precision of 98.95%. The output error is mainly attributed to nonideal factors in wiring and circuit components during the hardware experiment. Nevertheless, the hardware experimental results are generally consistent with the PSpice simulation results, demonstrating the practical feasibility of the circuit.

### B. Analysis of the Impact of Interconnect Resistance on the Proposed Circuit

In this section, fixed resistors are inserted between every two adjacent rows and columns in the cross-point array of the TINV to emulate the impact of interconnect resistance on circuit performance [35]. The effect of interconnect resistance on the circuit is interpreted as a perturbation, and the magnitude of the perturbation is positively correlated with the interconnect resistance value and the array size [36]. TINV's

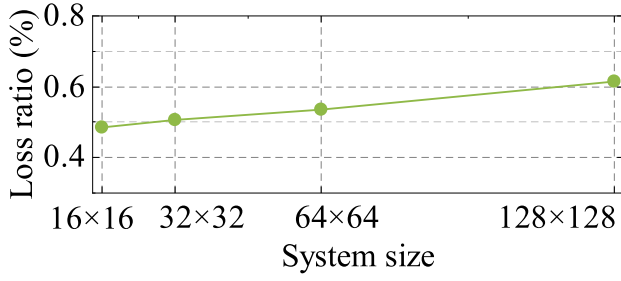


Fig. 10. Average accuracy loss ratio of TINV under interconnect resistance ( $1.5 \Omega$ ) for the system (1) of different sizes with the same condition number (40).

response to the perturbation depends on the condition number of the coefficient matrix  $A$  in system (1). The experiment first evaluates the impact of varying interconnect resistance values on TINV, with resistance treated as the variable. Specifically, based on the International Technology Roadmap for Semiconductors [37], interconnect resistances of 1, 1.5, 3, and  $4.5 \Omega$  are selected as test cases for the experiment. As shown in Fig. 9(a), the accuracy loss ratio of TINV is positively correlated with the interconnect resistance values. When the interconnect resistance in the circuit ranges from 1 to  $1.5 \Omega$ , the average accuracy loss ratio of the tested circuits remains within 0.25%. These results correspond to the 65-nm technology node. For higher technology nodes, TINV suffers average accuracy losses of 0.75%, 0.44%, 0.81%, and 1.01% when processing system (1) with sizes of  $16 \times 16$ ,  $32 \times 32$ ,  $64 \times 64$ , and  $128 \times 128$  under a  $4.5 \Omega$  interconnect resistance. The differences arise partly from the varying system (1) sizes in the four sets of experiments, and partly from the different condition numbers associated with each size. In Fig. 9(b), the accuracy loss of TINV is further evaluated by treating the condition number of matrix  $A$  in system (1) as the variable. The results show that an increase in the condition number significantly amplifies the perturbation caused by interconnect resistance, thereby leading to greater accuracy loss. In contrast, as shown in Fig. 10, the increase in accuracy loss ratio caused by the change in system (1) size (i.e., cross-point array scale) is more gradual. When the system size increases from  $16 \times 16$  to  $128 \times 128$ , the accuracy loss ratio rises from 0.48% to 0.61%. Nevertheless, all cases demonstrate average accuracies above 97%.

### C. Analysis of the Impact of Device Variation on the Proposed Circuit

This part of the experiment was conducted using the Monte Carlo analysis tool in the PSpice simulation platform, with a total of 80 runs. Device parameter variations were modeled using a Gaussian distribution. The tolerance values of the cross-point array were gradually increased to identify worst case scenarios. The scatter plot in Fig. 11 illustrates the average accuracy of each run under different device variations and system sizes. The dotted line plot then represents the average of all results. As the device variation increases, the results become progressively more scattered. An accuracy

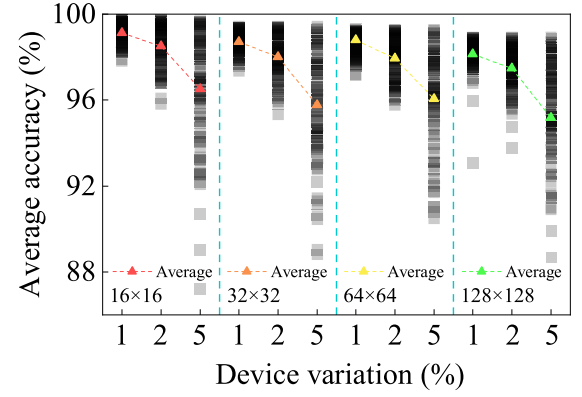


Fig. 11. Distribution of the average accuracy of TINV under the influence of device variation.

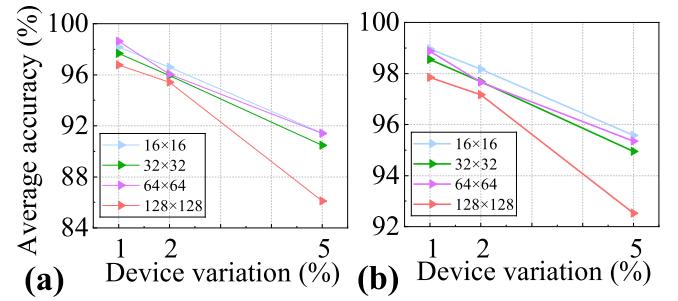


Fig. 12. Average accuracy of TINV under the influence of device variation. (a) Statistical result at three standard deviations. (b) Statistical result at two standard deviations.

drop below 90% is first observed when the device variation reaches 5%. With low device variations, most of the results cluster near the top of the axis, and the average accuracy approaches 98%. In addition, the output of the circuit approximately follows a normal distribution. The average accuracy calculated at three and two standard deviations is presented in Fig. 12(a) and (b), respectively. For device variation of 1%–2%, approximately 95% of the test results exhibit average accuracies exceeding 96.60%, 95.93%, 96.02%, and 95.41%, respectively. Meanwhile, 68% of the test results achieve accuracies exceeding 98.17%, 97.68%, 97.66%, and 97.17%, respectively. In TINV, the nonideal effects caused by device variations can also be interpreted as perturbations introduced into the system. The ratio of device variation reflects the magnitude of the perturbation. However, the direction of the perturbation affecting an individual device is random, leading to uncertainty in the accuracy variation caused by device deviations. Fig. 13(a) examines the accuracy variation of TINV with system size as the variable, and the results show that the accuracy variation is positively correlated with system size. In addition, the condition number of matrix  $A$  in system (1) also affects the uncertainty of accuracy variation. As shown in Fig. 13(b), the average coefficient of variation  $\left( (1/n) \times \sum_{j=1}^n (\sigma_{x^*,j} / \mu_{x^*,j}) \times 100\% \right)$ ,  $\sigma_{x^*,j}$  and  $\mu_{x^*,j}$  denote the mean and standard deviation, respectively, of  $x^*,j$ , is used as an evaluation metric to reflect the average fluctuation of output components. The results show that the output fluctuation

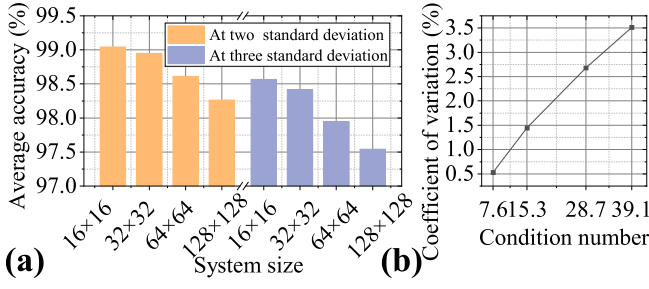


Fig. 13. (a) Average accuracy of TINV under identical device variation (2%) for the system (1) of different sizes with the same condition number (7.6). (b) Average coefficient of variation of TINV output components under device variation (2%) for same-sized ( $32 \times 32$ ) system (1) with different condition numbers.

increases as the condition number increases. A key concern is that increased uncertainty in accuracy variation indicates a higher probability of errors in individual computations. Although system size, condition number, and device variation ratio all affect the uncertainty of accuracy variation, the most effective way to reduce this uncertainty is to use devices with smaller variations. Alternatively, when high device deviations occur in real circuits, it may be necessary to implement accuracy compensation measures or preprocessing before compilation for high-precision computational requirements [38].

#### D. Circuit Performance Analysis Under Noise Interference

This section simulates the impact of noise on TINV performance. Thermal noise, shot noise, OA noise, and external input noise are included as indicated. Thermal noise and shot noise are modeled as current sources connected in parallel with each memristor of the cross-point resistive memristor array [39] and are given by

$$i_{\text{rms}} = \sqrt{Gf(4K_B T + 2qV)} \quad (24)$$

where  $G$  denotes the conductance of the memristor,  $f$  is the operating frequency,  $K_B$  represents the Boltzmann constant,  $T$  is the absolute temperature in Kelvin,  $q$  is the elementary charge, and  $V$  corresponds to the voltage across the two terminals of the memristor. The OA noise is modeled as two equivalent input current noise sources and one equivalent input voltage noise source [40]. The external noise is modeled as Gaussian white noise applied to the input of the TINV, simulating input noise from external sources such as a DAC.

The experiment considers a  $16 \times 16$  system (1). In the TINV, the mapping of the coefficient matrix  $A$  is shown in Fig. 14(a). Fig. 14(b) shows the mapping of the right-hand side vector  $b$ , that is, the input voltage of the TINV. The dynamic output response of the TINV under different noise levels, obtained from PSpice simulations, is shown in Fig. 14(c). The control group includes no noise at all. The experimental groups simultaneously include internal noise (thermal noise, shot noise, and OA noise) and external input noise at levels of 0%, 1%, 2%, and 5%. The simulation results show that with the addition of noise, the output of the TINV fluctuates around the ideal value. This fluctuation becomes

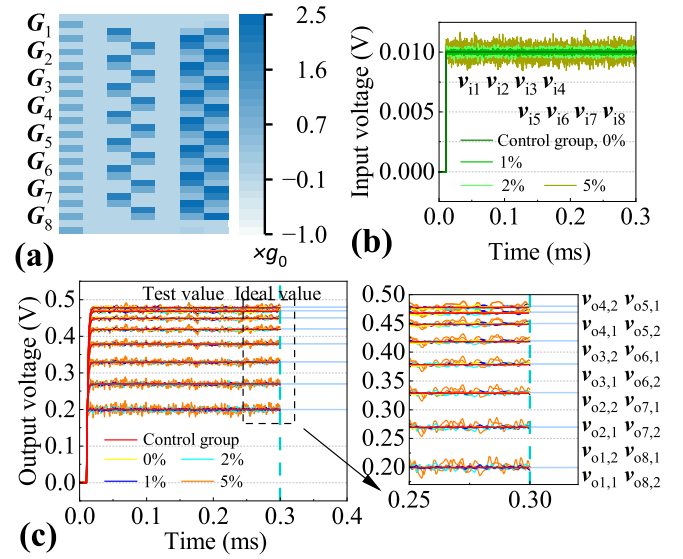


Fig. 14. Circuit performance under noise interference. (a) Mapping of coefficient matrix  $A$  of system (1). (b) Input voltage of the TINV. (c) Dynamic output response of the TINV under different noise levels.

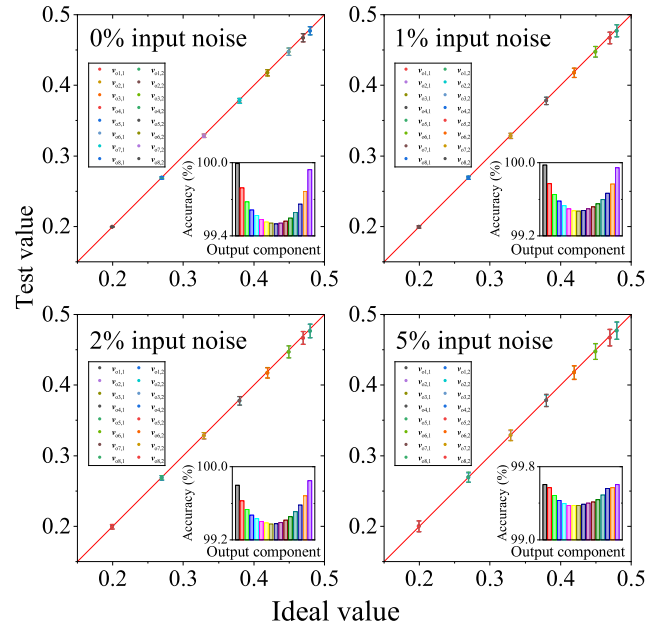


Fig. 15. Mean and standard deviation of output components under different input noise levels, and accuracy at mean points.

more pronounced as the level of input noise increases. In the experiment, the standard deviation is used to evaluate the fluctuation level of the output, while the accuracy at the mean point is used to characterize the solving precision under noise influence [41]. Fig. 15 presents the mean and standard deviation of each output component after the TINV reaches a stable output, as well as the accuracy corresponding to each mean value. All four groups achieve average accuracy above 99%. The maximum standard deviations are 0.0058 (0% input noise), 0.0083 (1% input noise), 0.0095 (2% input noise), and 0.0122 (5% input noise). Although the accuracy at the mean value shows promising results, the standard deviation deserves

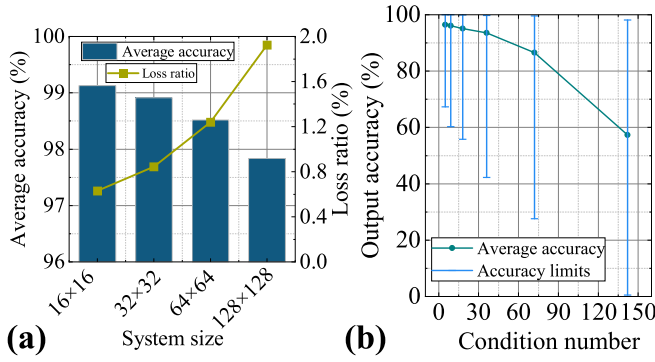


Fig. 16. (a) Average accuracy and accuracy loss ratio of TINV solving systems of different scales with the same condition number (3.6) under combined nonideal factors. (b) Accuracy statistics of TINV for a fixed system size under varying condition numbers.

more attention. It reflects the circuit's noise immunity [42]. Maintaining a low level of input noise is more favorable for stable circuit operation.

#### E. Circuit Performance Evaluation Under Combined Influencing Factors

This section evaluates the circuit performance under combined influencing factors. Specifically, the circuit simulation simultaneously includes  $1.5\ \Omega$  interconnect resistance, 2% device variation, and noise factors (thermal noise, shot noise, OA noise, and 2% Gaussian white noise). The experimental variables include system size and condition number. The experiment first tests the average accuracy of TINV under the influence of combined nonideal factors with system size as the variable. As shown in Fig. 16(a), the average accuracy of TINV decreases with increasing system size, and the accuracy loss ratio is positively correlated with system size. Under the combined influence of all nonideal factors, the average accuracy falls below 98% for the first time when the size reaches  $128 \times 128$ . Fig. 16(b) presents the accuracy statistics of TINV for systems of the same size with different condition numbers. Average accuracy and accuracy limits are taken as the subjects of analysis. The analysis focuses on average accuracy and accuracy limits. From the average accuracy curve, when the condition number is less than 45, the average accuracy of TINV remains above 95%. However, as the condition number increases, the lower bound of TINV decreases. When the condition number exceeds 30, the accuracy of some components falls below 50%. Near a condition number of 150, TINV's accuracy severely deteriorates, with some results reaching zero accuracy, indicating solution failure. Overall, as the condition number increases, TINV becomes increasingly sensitive to nonideal factors, and the error magnitude rises accordingly. Therefore, for TLS problems with high condition numbers, reducing the impact of nonideal factors is of critical importance. For severely ill-conditioned matrices, regularization may be required [43]. However, in scientific computing, TLS structures are typically strictly diagonally dominant symmetric matrices, which helps ensure the stability of TINV solutions to some extent.

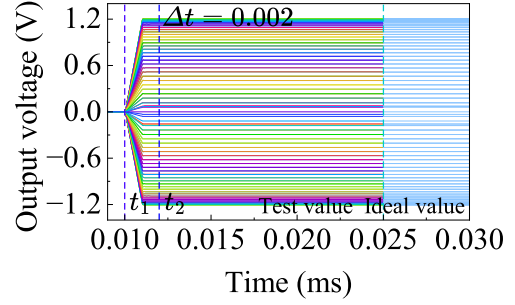


Fig. 17. Output waveform of system (1) with size  $128 \times 128$  solved by the VLTINV.

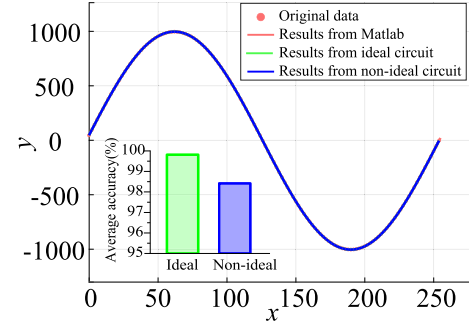


Fig. 18. Cubic spline curves constructed, respectively, from MATLAB output, ideal circuit output, and circuit output under comprehensive nonideal factors.

#### F. Circuit Performance Evaluation of VLTINV

This section considers the problem of solving a tridiagonal matrix equation arising from cubic spline interpolation [44], employing the VLTINV for computation. The parameters in the equation are derived from 128 uniformly spaced nodes on a sine wave. That is, the size of system (1) in this case is  $128 \times 128$ . And the group number  $p$  is set to 8. The experiments are conducted under both ideal and nonideal conditions. The nonideal conditions include  $1.5\ \Omega$  interconnect resistance, 2% device variation, thermal noise, and shot noise. In addition, to reflect practical application environments, 2% Gaussian white noise is added to the input of the circuit. The output under ideal conditions is shown in Fig. 17. The circuit reaches a steady state in approximately 0.002 ms, with all output components closely approaching the ideal values. The cubic spline curve constructed from the output of the circuit is shown in Fig. 18, and the control group is provided by the MATLAB cubic spline interpolation function. The final results demonstrate that an average accuracy of 99.82% is achieved under ideal conditions, whereas the accuracy under nonideal conditions reaches 98.41%. This also indicates that partitioned solving of large-scale systems can provide higher accuracy.

#### G. Estimation of Area Overhead and Power Consumption

The area and power overheads of the proposed circuit are evaluated using a 65-nm technology node, focusing primarily on the main computational circuitry, interconnects, and transmission gates, which are not included [45]. For TINV, the area of a subregion  $U$ , denoted as  $S_U$ , is used as the unit of measurement. It includes two OAs and a  $4 \times 7$  memristor

TABLE I  
COMPARISON OF REGION PARTITIONING AND FULL-SCALE MAPPING SCHEMES

| Category                          | Full-size mapping | This work          | Percentage reduction (%)     |
|-----------------------------------|-------------------|--------------------|------------------------------|
| Number of devices                 | $2n(n+1)$         | $14(n+1)$ or $14n$ | $(1 - 7/n)$ or $(n-6)/(n+1)$ |
| Number of Interconnect resistance | $2n(n+1)$         | $14(n+1)$ or $14n$ | $(1 - 7/n)$ or $(n-6)/(n+1)$ |

TABLE II  
COMPARISON OF THIS WORK WITH RELATED STUDIES

| Methods           | Steps and speed of core computation | Block | Remove zero regions | Scalability        |
|-------------------|-------------------------------------|-------|---------------------|--------------------|
| CPU [1]           | Gradual elimination (ms)            | —     | —                   | —                  |
| GPU [1]           | Parallel cyclic reduction (ms)      | —     | —                   | —                  |
| CC-Based INV [20] | One step ( $\mu$ s)                 | No    | —                   | —                  |
| BlockAMC [19]     | One step ( $\mu$ s)                 | Yes   | No                  | Stage solver       |
| This work         | One step ( $\mu$ s)                 | Yes   | Yes                 | Parallel Expansion |

array. Accordingly, for a system (1) of size  $n \times n$  mapped onto TINV, the total area is  $S_{\text{TINV}} = nS_U/2$ . The reference power consumption for solving the system is

$$P_{\text{TINV}} = \sum_{i=1}^{n/2} (\sum_{j=1}^2 g_0 v_{i,j}^2 + \sum_{j=1}^7 G_{i,jk} v_{Gi,jk}^2) + nV_{\text{OA}} I_q. \quad (25)$$

Here,  $G_{i,jk}$  denotes the conductance at the  $(j, k)$  position of the memristor array in  $U_i$ , and  $v_{Gi,jk}$  is the voltage applied across it.  $v_{i,j}$  represents the voltage across the reference conductance  $g_0$  connected to  $v_{ii,j}$ .  $V_{\text{OA}}$  is the supply voltage of the OAs, and  $I_q$  (100  $\mu$ A [46]) is its static bias current. For a large-scale  $N \times N$  system (1) decoupled into  $p$  groups and solved using VLTINV. The corresponding area overhead is  $S = S_U(fN/2p + p + f(N/p - 2)/2)$ , and the power consumption is  $P = 3\sum_{i=1}^p P_{\text{TINV}i} + P_{\text{TINV}\hat{S}} + \sum_{i=1}^p P_{\text{MVM}i}$ .  $P_{\text{TINV}i}$ ,  $P_{\text{TINV}\hat{S}}$ , and  $P_{\text{MVM}i}$  are derived from FINV, PINV, and SRE, respectively, and are calculated according to (25). Fig. 19 illustrates the area and power overheads corresponding to the system sizes evaluated in this section.

#### H. Comparison With Existing Solvers

This section discusses the advantages of the proposed circuit in solving large-scale sparse TLS problems. Table I compares the proposed region partitioning and zero-region removal scheme with the full-scale mapping scheme. The comparison covers device overhead and the number of interconnect resistances. For an  $n \times n$  TLS, the full-scale mapping scheme requires  $2n(n+1)$  devices. It also involves  $2n(n+1)$  interconnect resistances. By contrast, the proposed scheme requires  $14(n+1)$  devices for odd  $n$ , or  $14n$  devices when  $n$  is even. The number of interconnect resistances equals the number of devices. The number of devices and interconnect resistances is reduced by  $(1 - 7/n)$  (for odd  $n > 7$ ) or  $(n-6)/(n+1)$  (for even  $n > 6$ ). Additionally, nonideal sources originating from the devices themselves, such as device variation and noise, are mitigated to a similar extent. Therefore, compared with the full-scale mapping scheme, the contribution of the proposed region partitioning and zero-region removal scheme is to reduce the impact of nonideal factor sources on circuit solving accuracy.

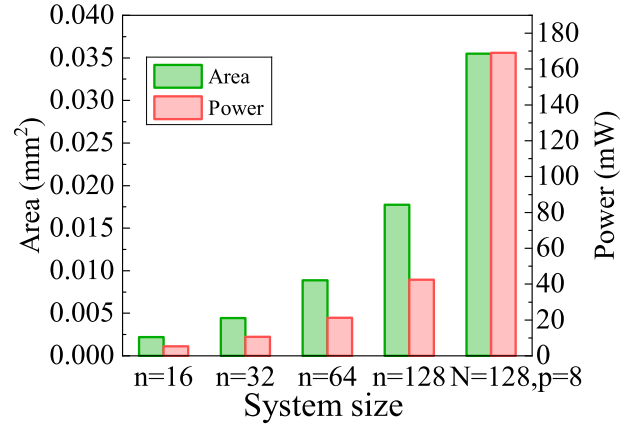


Fig. 19. Area and power overhead corresponding to the system sizes evaluated in this section.

Table II illustrates a comparison between this work and related studies. In comparison with [19] and [20], this work highlights its scalability and compactness advantages in solving large-scale sparse TLS problems. However, it should be noted that this work presents a circuit specifically designed for solving TLS, [19] and [20] are intended for general-purpose matrices. For the comparison of computation time, it should be noted that the benchmark targets the core computation in the TLS solving process. Additional overhead time is not included. This excludes data read/write during CPU and GPU processing, as well as DAC/ADC conversions and macro execution time in circuits. Furthermore, computation in one step should be understood as a calculation that does not require iterative or elimination operations during the solving process. The operation of the circuit is not completed in one step, and additional processes such as conductance writing and DAC/ADC conversions are also involved. With reference to existing technology, memristor array programming time can be completed within microseconds [47]. The total delay of ADC and DAC is less than 12  $\mu$ s [14]. The computation time of TINV is on the microsecond scale, limited by the bandwidth of the OAs and the condition number of the

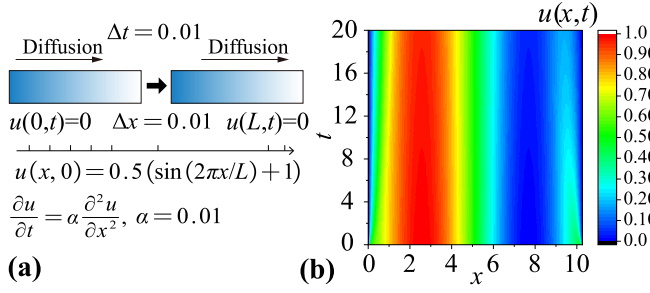


Fig. 20. Discussion on the application of VLTINV to solving a 1-D diffusion equation. (a) Diffusion model under consideration. (b) Simulation result provided by MATLAB at  $t \in [0, 20]$ .

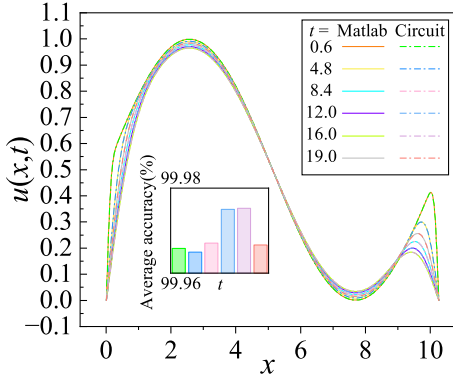


Fig. 21. Comparison between the proposed circuit-based solution and the MATLAB-provided results, along with the corresponding accuracy.

coefficient matrix, and independent of the matrix size [48]. In contrast, the computational complexities of solving TLS with the Gaussian elimination algorithm [1] are  $O(n)$  on CPU and  $O(\log n)$  on GPU, and the runtime is constrained by the TLS size. Moreover, compared to solving TLS using CPU or GPU, the INV approaches [20] offer a two-order-of-magnitude speed advantage. The experimental results in Section III-F also demonstrate that this work achieves the same benefit. This phenomenon can be attributed to the fact that the INV bypasses most iterative steps and data exchanges, solving one-step parallel computation.

#### IV. APPLICATION OF THE VLTINV CIRCUIT IN SOLVING DIFFUSION EQUATIONS

Consider a boundary value problem for a 1-D diffusion equation

$$\begin{aligned} \frac{\partial u}{\partial t} &= \alpha \frac{\partial^2 u}{\partial x^2} \\ \text{s.t.:} \\ u(x, 0) &= \varphi(x), \quad 0 < x < L \\ u(0, t) &= \mu_1(t), \quad 0 \leq t \leq T \\ u(L, t) &= \mu_2(t), \quad 0 \leq t \leq T \end{aligned} \quad (26)$$

where  $\alpha$  denotes the diffusion coefficient. The implicit finite difference scheme yields [49]

$$\frac{u_i^{k+1} - u_i^k}{\Delta t} = \alpha \frac{u_{i+1}^{k+1} - 2u_i^{k+1} + u_{i-1}^{k+1}}{\Delta x^2}. \quad (27)$$

Here,  $\Delta t$  and  $\Delta x$  denote the temporal and spatial step sizes, respectively.  $u_i^k$  represents the finite difference approximation of  $u(x_i, t_k)$ . Let  $r = \alpha(\Delta t / \Delta x^2)$ , then (28) can be expressed as the following tridiagonal matrix equation:

$$\mathbf{A} \mathbf{u}^{k+1} = \mathbf{u}^k$$

$$\mathbf{A} = \begin{bmatrix} 1+2r & -r & & & \\ -r & 1+2r & -r & & \\ & \ddots & \ddots & \ddots & \\ & & -r & 1+2r & -r \\ & & & -r & 1+2r \end{bmatrix}. \quad (28)$$

This section considers a diffusion model shown in Fig. 20(a), where  $L = 12.5$ ,  $\Delta t = 0.01$ , and  $\Delta x = 0.01$ . The boundary conditions are  $u(0, t) = 0$  and  $u(L, t) = 0$ , and the initial condition is  $u(x, 0) = 0.5(\sin(2\pi x/L) + 1)$ . The coefficient matrix  $\mathbf{A}$  obtained from the difference format is of size  $1024 \times 1024$ . Fig. 20(b) shows the simulation result at  $t \in [0, 20]$ , obtained using MATLAB. Six time points were selected and solved using the VLTINV. For the decomposition of matrix  $\mathbf{A}$ , the number of groups  $p$  is set to 8. Taking the MATLAB simulation results as the reference group, the average accuracy of the circuit outputs exceeds 99%, as shown in Fig. 21.

#### V. CONCLUSION

This article proposes a dedicated INV for solving TLS. It overcomes the low hardware efficiency and poor load-handling capability of general-purpose INV when solving large-scale sparse TLS problems. The basic block circuit that retains only the nonzero region has a more compact structure. Its stability is not affected by the zero regions. The basic block-based design fully decouples the preprocessing of VLTLS and supports vertical scalability. The SRE is also performed entirely in the analog domain. Simulation results show that the proposed circuit achieves 99% solving accuracy and provides solution outputs within the microsecond range. Under the influence of a combination of nonideal factors, the accuracy decreases by only 1.41%, maintaining 98.41%. The proposed circuit effectively solved large-scale difference equations ( $1024 \times 1024$ ) that arose from the numerical treatment of one-dimensional diffusion problems. We expect this compact and decouplable INV to provide targeted solutions for the significantly sparse and large-scale TLS generated by various computational tasks.

#### REFERENCES

- [1] S. Širca and M. Horvat, *Computational Methods in Physics* (Graduate Texts in Physics), 3rd ed., Cham, Switzerland: Springer, 2025.
- [2] K. Atkinson, W. Han, and D. E. Stewart, *Numerical Solution of Ordinary Differential Equations*. Hoboken, NJ, USA: Wiley, 2009.
- [3] G. Wolberg, *Cubic Spline Interpolation: A Review*. New York, NY, USA: Columbia University, 1988.
- [4] B. L. B. Liu, L. Z. L. Zhu, Z. Y. Z. Yang, and G. H. G. He, "Quantum implementation of numerical methods for convection-diffusion equations: Toward computational fluid dynamics," *Commun. Comput. Phys.*, vol. 33, no. 2, pp. 425–451, Jan. 2023.
- [5] J. Chung, S. Kang, H.-Y. Kim, and Y.-J. Kim, "Emergence of boundary conditions in the heat equation," *J. Math. Phys.*, vol. 65, no. 7, Jul. 2024, Art. no. 071501.

- [6] Y. Liu and Q. Huang, "Inverter dynamic response simulation via neural ordinary differential equations," in *Proc. Int. Workshop Artif. Intell. Mach. Learn. Energy Transformation (AIE)*, May 2024, pp. 1–4.
- [7] L. Angheluta, *Ordinary Differential Equations*. Cham, Switzerland: Springer, 2025, pp. 97–146.
- [8] P. Wan and Z. Zeng, "Point-based stabilization control of linear parabolic partial differential equation systems on space–time scales," *IEEE Trans. Autom. Control*, vol. 70, no. 4, pp. 2552–2559, Apr. 2025.
- [9] S. Williams and J. Deshmukh, "Potential games on cubic splines for self-interested multi-agent motion planning," *IEEE Control Syst. Lett.*, vol. 8, pp. 2487–2492, 2024.
- [10] M. A. Zidan et al., "A general memristor-based partial differential equation solver," *Nature Electron.*, vol. 1, no. 7, pp. 411–420, Jul. 2018.
- [11] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA, USA: SIAM, 2006.
- [12] Y. Saad, *Iterative Methods for Sparse Linear Systems*. Philadelphia, PA, USA: SIAM, 2003.
- [13] C. Zhang, H. Sun, S. Li, Y. Wang, H. Chen, and H. Liu, "A survey of memory-centric energy efficient computer architecture," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 10, pp. 2657–2670, Oct. 2023.
- [14] T. Song, X. Chen, and Y. Han, "Eliminating iterations of iterative methods: Solving large-scale sparse linear system in  $O(1)$  with RRAM-based in-memory accelerator," in *Proc. Great Lakes Symp. VLSI*. New York, NY, USA: Association for Computing Machinery, Jun. 2021, pp. 71–76.
- [15] J. Li et al., "Fully analog iteration for solving matrix equations with in-memory computing," *Sci. Adv.*, vol. 11, no. 7, p. 6391, Feb. 2025.
- [16] Z. Sun, G. Pedretti, E. Ambrosi, A. Bricalli, W. Wang, and D. Ielmini, "Solving matrix equations in one step with cross-point resistive arrays," *Proc. Nat. Acad. Sci. USA*, vol. 116, no. 10, pp. 4123–4128, Mar. 2019.
- [17] P. Mannonci and D. Ielmini, "A generalized block-matrix circuit for closed-loop analog in-memory computing," *IEEE J. Explor. Solid-State Comput. Devices Circuits*, vol. 9, pp. 47–55, 2023.
- [18] Q. Hong, S. Man, J. Sun, S. Du, and J. Zhang, "Programmable in-memory computing circuit for solving combinatorial matrix operation in one step," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 7, pp. 2916–2928, Jul. 2023.
- [19] L. Pan, P. Zuo, Y. Luo, Z. Sun, and R. Huang, "BlockAMC: Scalable in-memory analog matrix computing for solving linear systems," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2024, pp. 1–6.
- [20] Y. Luo, P. Zuo, S. Wang, Z. Sun, and R. Huang, "Smaller, faster, lower-power analog RRAM matrix computing circuits without performance compromise," *Sci. China Inf. Sci.*, vol. 68, no. 2, Feb. 2025, Art. no. 122402.
- [21] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. Philadelphia, PA, USA: SIAM, 2022.
- [22] Z. Sun, G. Pedretti, P. Mannonci, E. Ambrosi, A. Bricalli, and D. Ielmini, "Time complexity of in-memory solution of linear systems," *IEEE Trans. Electron Devices*, vol. 67, no. 7, pp. 2945–2951, Jul. 2020.
- [23] X. Liao, S. Li, Y. Lu, and J. E. Roman, "A parallel structured divide-and-conquer algorithm for symmetric tridiagonal eigenvalue problems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 2, pp. 367–378, Feb. 2021.
- [24] S. N. Truong and K.-S. Min, "New memristor-based crossbar array architecture with 50-% area reduction and 48-% power saving for matrix-vector multiplication of analog neuromorphic computing," *J. Semicond. Technol. Sci.*, vol. 14, no. 3, pp. 356–363, Jun. 2014.
- [25] Z. Sun, G. Pedretti, and D. Ielmini, "Fast solution of linear systems with analog resistive switching memory (RRAM)," in *Proc. 4th IEEE Int. Conf. Rebooting Comput.*, Mar. 2019, pp. 1–5.
- [26] V. Kalantzis et al., "Solving sparse linear systems via flexible GMRES with in-memory analog preconditioning," in *Proc. IEEE High Perform. Extreme Comput. Conf. (HPEC)*, Sep. 2023, pp. 1–7.
- [27] Y. Luo, S. Wang, P. Zuo, Z. Sun, and R. Huang, "Modeling and mitigating the interconnect resistance issue in analog RRAM matrix computing circuits," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 11, pp. 4367–4380, Nov. 2022.
- [28] G. Pedretti and D. Ielmini, "In-memory computing with resistive memory circuits: Status and outlook," *Electronics*, vol. 10, no. 9, p. 1063, Apr. 2021.
- [29] Å. Björck, *Numerical Methods in Matrix Computations*, vol. 59. Cham, Switzerland: Springer, 2015.
- [30] Z. Lu et al., "An RRAM-based computing-in-memory architecture and its application in accelerating transformer inference," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 3, pp. 485–496, Mar. 2024.
- [31] E. Polizzi and A. Sameh, "SPIKE: A parallel environment for solving banded linear systems," *Comput. Fluids*, vol. 36, no. 1, pp. 113–120, Jan. 2007.
- [32] P. Xiao, S. Du, Z. Wei, Q. Hong, and S. Wen, "Design and application of programmable analog computing circuit for Kalman filter algorithm based on memristive array," *IEEE Trans. Circuits Syst. Artif. Intell.*, vol. 1, no. 2, pp. 272–284, Dec. 2024.
- [33] Y. Li, L. Xie, P. Xiao, C. Zheng, and Q. Hong, "Drift speed adaptive memristor model," *Neural Comput. Appl.*, vol. 35, no. 19, pp. 14419–14430, Jul. 2023.
- [34] P. Xiao et al., "A Riccati matrix equation solver design based neuro-dynamics method and its application," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 15163–15176, 2025.
- [35] T. Cao, C. Liu, Y. Gao, and W. L. Goh, "Parasitic-aware modeling and neural network training scheme for energy-efficient processing-in-memory with resistive crossbar array," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 12, no. 2, pp. 436–444, Jun. 2022.
- [36] Y. Liao et al., "Diagonal matrix regression layer: Training neural networks on resistive crossbars with interconnect resistance effect," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 40, no. 8, pp. 1662–1671, Aug. 2021.
- [37] *International Technology Roadmap for Semiconductors*, Semiconductor Industry Association, Washington, DC, USA, 2013.
- [38] Q. Hong, H. Jiang, P. Xiao, S. Du, and T. Li, "A parallel computing scheme utilizing memristor crossbars for fast corner detection and rotation invariance in the ORB algorithm," *IEEE Trans. Comput.*, vol. 74, no. 3, pp. 1–14, Mar. 2025.
- [39] Z. He, J. Lin, R. Ewetz, J.-S. Yuan, and D. Fan, "Noise injection adaptation: End-to-end ReRAM crossbar non-ideal effect adaption for neural network mapping," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, Jun. 2019, pp. 1–6.
- [40] G. Giusi, F. Crupi, C. Pace, and P. Magnone, "Full model and characterization of noise in operational amplifier," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 56, no. 1, pp. 97–102, Jan. 2009.
- [41] S. Wang et al., "Optimization schemes for in-memory linear regression circuit with memristor arrays," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 12, pp. 4900–4909, Dec. 2021.
- [42] W. Zhang et al., "Edge learning using a fully integrated neuro-inspired memristor chip," *Science*, vol. 381, no. 6663, pp. 1205–1211, Sep. 2023. [Online]. Available: <https://www.science.org/doi/abs/10.1126/science.ade3483>
- [43] P. Mannonci, E. Melacarne, and D. Ielmini, "An analogue in-memory ridge regression circuit with application to massive MIMO acceleration," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 12, no. 4, pp. 952–962, Dec. 2022.
- [44] O. El Attiq, S. Reboul, A. Latif, and S. Abouelouafa, "High-precision satellite position interpolation: A comparison of Lagrange, cubic spline, and artificial neural network methods," in *Proc. Int. Microw. Antenna Symp. (IMAS)*, Oct. 2024, pp. 1–4.
- [45] P. Xiao, Q. Hong, S. Du, and J. Zhang, "Analog-in-memory accelerator design based on memristive arrays for opposite directional interference alignment algorithm," *IEEE Trans. Ind. Informat.*, vol. 20, no. 3, pp. 3628–3638, Mar. 2024.
- [46] H. Zumbahlen et al., *Linear Circuit Design Handbook*. London, U.K.: Newnes, 2011.
- [47] D. Sharma et al., "Linear symmetric self-selecting 14-bit kinetic molecular memristors," *Nature*, vol. 633, no. 8030, pp. 560–566, Sep. 2024, doi: [10.1038/s41586-024-07902-2](https://doi.org/10.1038/s41586-024-07902-2).
- [48] D. Bridarolli et al., "3-D vertical resistive switching random access memory (3D-VRRAM) with multilevel programming for high-density, energy-efficient in-memory computing," *IEEE Trans. Electron Devices*, vol. 72, no. 5, pp. 2677–2684, May 2025.
- [49] J. Randall, "Finite difference methods for differential equations," *A Math.*, vol. 585, no. 6, p. 112, 2005.